Guia de Flutter

Actualización del 24 de octubre de 2026: la aplicación web ya se compiló y probó. Las sesiones web se mantienen solo en memoria y las retenciones se obtienen de comprobantes importados. Las notas anteriores se conservan a continuación como historial; no ejecutar migrate dev ni flutter create para iniciar el proyecto existente. Para el arranque actual, consultar 01 Guia de inicio; para publicar, 07 Despliegue en Vercel.

Sistema Tributario RIMPE — Cliente Flutter (conectado al backend)

En Windows puedes iniciar la web y su API juntas con ../INICIAR.cmd y abrir http://localhost:3000. La carpeta web/ ya está incluida: no hace falta ejecutar flutter create . para la web. Consulta la guía actual en 01 Guia de inicio.docx; las notas de entorno al final corresponden a la preparación original.

App Flutter multiplataforma (Web + Android/iOS) para el backend NestJS del Sistema Tributario RIMPE. Ya no guarda datos de negocio localmente: todo vive en el servidor (auth, empresas, comprobantes, retenciones, declaraciones, motor de obligaciones y ATS). El dispositivo solo guarda el token de sesión.

Qué cambió respecto a la versión anterior (local-only)

Antes (v1, local)	Ahora (v2, conectada a la API)
StorageService con shared_preferences guardaba todo	Los datos viven en PostgreSQL, detrás del backend
El XML se parseaba en el dispositivo	El XML se sube tal cual y el backend lo parsea/clasifica
Un solo usuario, sin login	Login/registro con JWT, multiempresa, multiusuario
"Contribuyentes" locales	"Empresas" reales en el backend, con roles por usuario
Sin pantalla de obligaciones tributarias	Nueva pantalla "Mis obligaciones tributarias", consume el motor de reglas del backend

Antes de correrla: levanta el backend

Esta app no funciona sola — necesita el backend corriendo (ver el README.md del proyecto backend/). Resumen:

```
cd backend
cp .env.example .env
docker compose up -d db
npm install && npx prisma generate && npx prisma migrate dev --name init
npm run seed
npm run start:dev   # API en http://localhost:3000
```

Configurar la URL del backend en Flutter

Por defecto, lib/services/api_config.dart apunta a:

- http://localhost:3000 en Web y iOS/desktop.
- http://10.0.2.2:3000 en el emulador de Android (así es como el emulador ve la máquina anfitriona — no confundir con localhost, que dentro del emulador apunta al propio emulador).

Para apuntar a otra URL (un servidor real, o un dispositivo físico en la misma red), corre con --dart-define:

```
flutter run --dart-define=API_BASE_URL=http://192.168.1.50:3000
```

Puesta en marcha

```
flutter create .
```

```
flutter pub get
flutter run -d chrome      # o `flutter run` para Android
```

Al abrir la app por primera vez pedirá registrarte (correo + contraseña). Después de iniciar sesión, crea una empresa desde la pestaña "Empresas" — quedarás automáticamente como su ADMIN_EMPRESA.

Estructura

```
lib/
  main.dart                       Gate de autenticación + navegación
  models/                         DTOs que espejan las respuestas de la API
    company.dart, voucher.dart, tax_declaration.dart, obligacion.dart
  services/
    api_config.dart               URL base del backend por plataforma
    api_client.dart               HTTP + manejo de token + refresh automático
    api_exception.dart            Errores del backend con mensaje listo para UI
    session_storage.dart          Persistencia SOLO del token (no datos de negocio)
    auth_service.dart             Estado global de sesión (login/registro/logout)
    company_service.dart          GET/POST /api/companies
    voucher_service.dart          Import XML + listado de comprobantes
    declaration_api_service.dart  Generar/presentar declaración 104/103
    tax_obligations_service.dart  GET /api/tax/obligations
    ats_api_service.dart          Validar/generar/historial del ATS
  screens/
    login_screen.dart             Login y registro
    dashboard_screen.dart         Panel general
    companies_screen.dart         Alta y listado de empresas
    xml_import_screen.dart        Sube XML al backend (ya no lo parsea localmente)
    retenciones_screen.dart       Lista comprobantes con retención calculada
    declaraciones_screen.dart     104/103 + acciones de ATS (validar/generar/descargar)
    obligaciones_screen.dart      "Mis obligaciones tributarias" — motor de reglas
```

Qué NO se pudo probar aquí (léase antes de asumir que todo funciona)

Este entorno de trabajo no tiene el SDK de Flutter instalado, así que no se pudo compilar ni ejecutar la app para verificarla de punta a punta contra el backend real. Se revisó manualmente cada archivo (balance de llaves, imports, nombres de campos contra los DTOs reales que devuelve el backend), pero antes de darla por lista:

1. Corre flutter create . + flutter pub get para generar las carpetas nativas y bajar las dependencias.

2. Levanta el backend (ver arriba).

3. Prueba el flujo completo: registro → crear empresa → importar un XML de prueba → ver retención calculada → generar declaración → validar ATS.

Si algo no compila o un campo no coincide con lo que devuelve el backend, lo más probable es un desajuste de nombre entre un DTO de Flutter y la respuesta real de NestJS — son los primeros lugares a revisar.